

PROJETO PRÁTICO DE INTRODUÇÃO AOS ALGORITMOS

Sistema de Cadastro de Acontecimentos Históricos

Integrantes do grupo (Turma 10A):

- Alexsandro Ferreira
- Ana Júlia Gurita
- Carlos Vinicius Teixeira de Souza

1. Introdução:

Esse projeto teve como objetivo desenvolver um sistema de cadastro de acontecimentos históricos usando a linguagem C++. O sistema permite realizar operações básicas de gerenciamento, como cadastrar, consultar e remover registros. Além de salvar as informações em arquivos CSV para garantir que os dados não sejam perdidos ao fechar o programa. O objetivo principal do trabalho foi aplicar na prática conceitos de estrutura de dados. Entre eles, a alocação dinâmica de memória, a criação de vetores redimensionáveis, além de algoritmos de ordenação e de busca linear e binária. Tudo funciona no terminal por meio de um menu interativo no qual o usuário pode interagir com os acontecimentos históricos do mundo de forma simples.

2. Descrição das Estruturas e Lógica do Programa:

O programa foi organizado usando modularização, ou seja, foi dividido em várias funções fora da função principal, para facilitar a manutenção do código e deixá-lo mais organizado e legível.

2.1 Registro de Dados (struct Acontecimento):

Cada registro do sistema representa um acontecimento histórico e é armazenado em uma struct chamada **Acontecimento**. Essa estrutura guarda todas as informações necessárias de um evento:

- **ID:** número inteiro que identifica cada acontecimento de forma única
- **Nome:** nome do evento histórico
- **Local:** onde o acontecimento ocorreu
- **Ano:** ano em que o evento aconteceu (pode ser negativo para anos antes de Cristo)
- **Países envolvidos:** lista de países relacionados ao evento (em texto)
- **Removido:** variável booleana usada para indicar se o registro foi “apagado” logicamente.

Logo após a struct Acontecimento, tem as funções de manipulação de dados do nosso sistema de cadastro, são elas:

- **cria_acontecimento:** responsável por criar um novo registro de acontecimento histórico. Recebe as informações do evento (nome, local, ano e países envolvidos) e retorna uma estrutura já preenchida. O ID inicial é

definido como -1, indicando que o registro ainda não foi inserido na base de dados principal.

- **redimensiona_vetor:** responsável por aumentar a capacidade do vetor dinâmico quando ele atinge o limite máximo de armazenamento. O sistema utiliza um vetor dinâmico de Acontecimento, que começa com uma capacidade inicial e vai sendo preenchido conforme novos dados são adicionados. Quando essa capacidade é atingida, é necessário expandir o vetor para continuar armazenando informações. Para isso, a função cria um novo vetor com tamanho maior, copia todos os dados do vetor antigo para o novo e, em seguida, libera a memória ocupada pelo vetor anterior. Depois disso, o programa passa a utilizar o novo vetor redimensionado. Assim, o sistema cresce quando precisa, sem perder os dados e sem ter um limite fixo desde o início.
- **adiciona_acontecimento:** adiciona um novo acontecimento ao vetor. Antes da inserção, o sistema verifica se há espaço disponível e, caso necessário, chama a função de redimensionamento. Em seguida, o ID é atribuído automaticamente e os dados do novo registro são inseridos no vetor, aumentando o tamanho atual da estrutura.
- **remove_acontecimento:** realiza a remoção lógica de um registro. Em vez de apagar fisicamente o elemento do vetor, ela apenas marca o campo “removido” como verdadeiro. Isso evita reorganizações desnecessárias na memória. Caso o ID informado não seja encontrado, uma mensagem de erro é exibida.
- **string_para_int:** converte uma string numérica para um valor inteiro. Ela percorre cada caractere da string e realiza a conversão manual, simulando o funcionamento de uma transformação de texto em número.
- **gravar_modificacao_no_csv:** responsável por salvar os dados atualizados em um arquivo CSV. Percorre o vetor e grava apenas os registros que não foram removidos logicamente. Os dados são organizados em formato de tabela, separados por vírgulas, garantindo compatibilidade com arquivos CSV.
- **imprime_vetor:** exibe na tela as informações de um acontecimento específico. Verifica se o registro não foi removido e, em seguida, imprime todos os seus campos de forma organizada para o usuário.
- **ordena_por_ano:** organiza os acontecimentos em ordem crescente de ano utilizando um algoritmo de ordenação do tipo Shell Sort. Essa ordenação facilita a visualização cronológica dos acontecimentos e deixa os registros ordenados por ano.
- **ordena_por_id:** ordena os registros com base no ID do acontecimento, também utilizando Shell Sort. Essa ordenação facilita buscas diretas por identificador único.
- **buscaBinaria_por_id:** realiza uma busca binária pelo ID do acontecimento. Como o ID é único, a função retorna apenas um registro correspondente ao valor procurado, caso ele exista.

- **busca_por_ano:** realiza uma busca por intervalo de anos. Percorre todo o vetor e exibe os acontecimentos que possuem ano dentro do intervalo informado pelo usuário.
- **busca_por_intervalo_id:** realiza uma busca por intervalo de IDs. Percorre o vetor e exibe os acontecimentos ativos cujo ID esteja dentro do intervalo informado pelo usuário.
- **buscaLinear_por_nome:** realiza uma busca sequencial pelo nome do acontecimento. O vetor é percorrido elemento por elemento até encontrar o primeiro registro com nome correspondente ao informado.
- **mostra_trecho_por_posicao:** exibe um trecho do vetor de acordo com as posições inicial e final informadas pelo usuário, permitindo visualizar apenas uma parte dos registros armazenados.

3. Ordem e Formatação dos Dados Armazenados:

Os dados do nosso sistema são armazenados em arquivos no formato CSV. O arquivo de entrada inicial se chama `acontecimentos_historicos.csv`, e as alterações feitas pelo usuário são salvas em um novo arquivo chamado `novos_acontecimentos_historicos.csv`.

A estrutura do arquivo segue sempre a mesma ordem de colunas, definida no início do projeto. A primeira linha é um cabeçalho com os nomes dos campos, e as demais linhas contêm os dados dos acontecimentos. A ordem das informações no cabeçalho é: `id`, `nome`, `local`, `ano`, `paises_envolvidos`.

Observações:

- Quando um campo contém vírgulas dentro do texto, como na lista de países, ele é colocado entre aspas duplas. Isso evita que o programa entenda essas vírgulas como separação de colunas.
- Na leitura do arquivo, o programa lê cada linha do CSV e separa os dados corretamente. Os valores numéricos, como ID e ano, são convertidos para o tipo adequado usando funções de conversão de texto.

4. Acertos e Erros no Desenvolvimento:

Durante o desenvolvimento do projeto, uma das principais dificuldades foi a falta de uma organização e de um planejamento mais bem definido. Isso acabou dificultando a divisão de tarefas entre os membros da equipe e deixou o processo menos eficiente. Em alguns momentos, um integrante precisou fazer uma parte maior do trabalho sozinho e depois enviar para o repositório, o que não era o ideal. Essa situação mostrou que a ausência de uma melhor orquestração do projeto

acabou prejudicando a divisão equilibrada das tarefas e a colaboração entre o grupo.

Outra dificuldade foi que durante a junção dos códigos feitos por cada integrante, surgiram alguns problemas de duplicação de funções e erros de sintaxe. Também havia variáveis não declaradas em partes do redimensionamento do vetor. Esses problemas foram corrigidos depois, com revisões feitas pela equipe para ajustar e organizar melhor o código.

Como limitações conhecidas do sistema, o parser utilizado para leitura do arquivo CSV foi desenvolvido especificamente para o formato adotado nesse projeto, uma vez que existem formas diferentes de representar, por exemplo, os acontecimentos históricos. Além disso, a validação das entradas fornecidas pelo usuário no menu é limitada, assumindo que os valores digitados possuem o tipo esperado para cada campo. Por fim, a busca por nome retorna apenas o primeiro acontecimento encontrado com o nome informado. Essa decisão foi considerada adequada para este projeto, uma vez que a base de dados foi construída assumindo que não existem acontecimentos históricos distintos com exatamente o mesmo nome.

5. Conclusão:

O projeto foi concluído com sucesso, cumprindo todos os requisitos solicitados. O sistema desenvolvido em C++ mostrou-se estável e eficiente, sendo capaz de rodar sem travamentos e de salvar e ler os dados do arquivo CSV corretamente.

A realização deste trabalho foi fundamental para o nosso grupo entender, na prática, como gerenciar a memória do computador manualmente e como funcionam os algoritmos de busca e ordenação por trás dos panos. Além do aprendizado técnico com a programação, o projeto também ajudou a equipe a evoluir na organização do código e no trabalho em grupo usando o GitHub.

6. Resultados Obtidos:

O sistema foi testado utilizando a base de dados do arquivo `acontecimentos_historicos` e apresentou os seguintes resultados práticos:

- **Manipulação de Arquivos:** O programa conseguiu ler perfeitamente todos os dados iniciais do arquivo e gerou o arquivo `novo_acontecimentos_historicos` com as alterações salvas de forma organizada.
- **Eficiência nas Buscas e Ordenação:** Mesmo rodando direto no terminal, as operações de ordenação, bem como as buscas, acontecem instantaneamente, sem travamentos ou lentidão.
- **Estabilidade da Memória:** O sistema de vetor redimensionável funcionou como esperado, aumentando a capacidade da memória automaticamente a cada novo cadastro sem gerar erros ou fechar o programa sozinho.

- Interface Funcional: O menu interativo se mostrou prático e intuitivo, permitindo navegar pelas as funções (cadastrar, buscar e remover) sem dificuldades.